Schema-Graph-Assisted Text-to-SQL for Materials Databases: A Method for Natural Language Querying of Normalized Relational Materials Data

— Validated on OQMD Intermetallic Compound Data (B2 and $L1_2$) —

Satoshi Minamoto
National Institute for Materials Science (NIMS)
1-2-1 Sengen, Tsukuba, Ibaraki 305-0047, Japan

Abstract

Materials databases such as the Open Quantum Materials Database (OQMD), Materials Project, and AFLOW have accumulated vast quantities of first-principles-calculated property data, yet accessing these data through normalized relational schemas requires SQL expertise that most materials scientists lack. We present a **Schema-Graph-Assisted Text-to-SQL (T2SQL) method** that automatically converts natural language queries into executable SQL statements for normalized materials databases.

The method comprises four key technical components: (1) a **domain-specific entity extractor** with a bilingual (Japanese/English) materials terminology dictionary, (2) a **Schema Graph Traversal Engine** that represents foreign key (FK) relationships as a NetworkX graph and computes minimal JOIN paths via Steiner tree approximation, (3) a **Few-Shot Retrieval-Augmented Generation (RAG) store** (SQL-as-Few-Shot-Examples) that accumulates successful NL–SQL pairs and retrieves similar examples via TF-IDF cosine similarity for LLM prompt injection, including automatic seed example extraction from LaTeX manuscripts, and (4) a **multi-layer SQL Guard** that enforces keyword blacklisting, single-statement restriction, table whitelisting, and automatic LIMIT injection.

Validation was performed on 909 OQMD intermetallic compounds (636 B2-type CsCl and 273 $L1_2$-type $AuCu_3$) across 39 test cases spanning normal queries, absent-data queries, ambiguous/malformed inputs, contradictory conditions, SQL injection attacks, and safety checks. A three-level comparison (Naive T2SQL without Schema Graph / Schema Graph + Rule-based / Schema Graph + GPT-5) demonstrated that Schema Graph traversal eliminates unnecessary JOINs, correctly handles multi-element AND queries (which yield zero results under the Naive approach), and achieves Precision = 1.0 against direct OQMD API ground truth for all validated test cases. The Rule-based mode achieves <100 ms latency with zero external API dependency, while the GPT-5 mode provides superior vocabulary coverage and numeric filter generation at the cost of ∼7.3 s latency and token consumption.

This method lowers the barrier for materials researchers to query normalized relational databases using natural language. In the emerging paradigm of AI for Science (AI4S), where AI is expected to accelerate every stage of the scientific workflow—from hypothesis generation through data retrieval to experimental design—the ability to access structured materials data via natural language constitutes critical infrastructure. This work contributes to the democratization of data utilization in materials informatics and provides a foundational building block for AI4S-driven materials discovery.

Keywords: Text-to-SQL, materials database, schema graph, Retrieval-Augmented Generation, OQMD, intermetallic compounds, natural language processing, materials informatics, AI for Science

Contents

1 Introduction

1.1 AI for Science and the Data Access Barrier in Materials Informatics

The paradigm of **AI for Science (AI4S)**—the systematic application of artificial intelligence to accelerate scientific discovery—is transforming materials research [Wang et al., 2023]. AI4S envisions an integrated workflow where AI agents autonomously generate hypotheses, retrieve relevant data from large-scale databases, design experiments or simulations, and interpret results. Within this vision, **natural language access to structured materials data** is not merely a convenience but a prerequisite: if AI agents (or the researchers directing them) cannot efficiently query normalized relational databases, the entire AI4S pipeline is bottlenecked at the data retrieval stage.

The rapid growth of high-throughput first-principles calculations has produced several large-scale materials databases: the Open Quantum Materials Database (OQMD, ∼1M entries) [Saal et al., 2013, Kirklin et al., 2015], the Materials Project (>150K entries) [Jain et al., 2013], AFLOW (>3.5M entries) [Curtarolo et al., 2012], and NOMAD [Draxl and Scheffler, 2018]. These databases are indispensable for computational materials discovery, screening of candidate phases [Senkov et al., 2018, George et al., 2019], and validation of experimental observations.

However, accessing data stored in normalized relational schemas (multiple tables connected by foreign keys) requires SQL expertise. For example, querying "stable B2 compounds containing Fe with their lattice constants" on a properly normalized schema demands a 4-table JOIN with WHERE conditions on element, prototype, and thermodynamic stability—a query that is straightforward for database engineers but non-trivial for materials scientists.

While REST APIs (such as OQMD's `/oqmdapi/formationenergy`) partially address this barrier, they impose their own learning curve (filter syntax, pagination handling, response parsing) and lack the flexibility of arbitrary SQL queries over locally curated datasets.

In the AI4S context, this data access barrier is particularly critical: autonomous AI agents performing materials screening or property prediction must issue complex, multi-condition database queries programmatically. A natural language interface that reliably generates correct SQL enables both human researchers and AI agents to interact with materials databases without database engineering expertise, thereby removing a key bottleneck in the AI4S workflow.

1.2 Text-to-SQL: State of the Art

Text-to-SQL (T2SQL) research has advanced rapidly with large-scale benchmarks: Spider [Yu et al., 2018] (10,181 questions, 200 databases, 138 domains) and BIRD [Li et al., 2024] (12,751 questions, 95 databases, 33.4 GB). Recent LLM-based methods achieve high accuracy: DAIL-SQL [Gao et al., 2023] reports 86.6% execution accuracy on Spider using skeleton-based few-shot example selection, and DIN-SQL [Pourreza and Rafiei, 2024] achieves 85.3% using decomposed in-context learning. Hong et al. [2024] provide a comprehensive survey noting the dominant role of schema linking. Maamari et al. [2024] argue that advanced LLMs may render explicit schema linking unnecessary, though this remains debated.

However, **all existing T2SQL benchmarks target general-purpose domains** (e-commerce, healthcare, education), and no prior work has addressed the specific challenges of materials science databases: domain-specific terminology (Strukturbericht designations, prototype names), bilingual queries (Japanese/English element names), composition table normalization (one row per element per compound), and thermodynamic stability filters ($E_{\text{hull}} \leq 0.001$ eV/atom).

1.3 NLP in Materials Informatics

NLP applications in materials science are expanding: MatSci-NLP [Song et al., 2023] benchmarks 7 NLP tasks on materials text, LLAMAT [Mishra et al., 2024] improves information extraction via continual pre-training of LLaMA, the Materials Knowledge Navigation Agent

(MKNA) [Zhuang and Farimani, 2026] translates natural language intent into database retrieval and structure generation actions, and OptiMat Alloys [Hu and Turlo, 2026] provides an LLM-powered conversational alloy explorer. Materials Project has begun offering MCP-based LLM integration.

These works address information extraction and agent-based database interaction, but **none target schema-aware SQL generation on normalized relational databases**—specifically, automatic JOIN path discovery through foreign key graph traversal. This gap is the focus of our method.

1.4 Objectives and Novelty

Our contributions are threefold:

1. **A materials-domain-specific Schema Graph T2SQL method** combining a bilingual materials terminology dictionary with FK relationship graph traversal for automatic JOIN path computation on normalized materials databases.
2. **SQL-as-Few-Shot-Examples with manuscript-based seed extraction**: a RAG feedback loop that accumulates successful NL–SQL pairs and retrieves them via TF-IDF similarity for LLM prompt injection, with automatic seed example extraction from LaTeX research papers.
3. **Integration test methodology using OQMD API ground truth**: quantitative validation of the entire data pipeline (API → CSV → RDB normalization → NL → SQL → JOIN reconstruction) using Precision/Recall against direct API queries.

2 Methods

This section describes each technical component in detail. The overall pipeline architecture is shown in Fig. 1.

Fig. 1: System Architecture — Schema-Graph-Assisted Text-to-SQL Pipeline

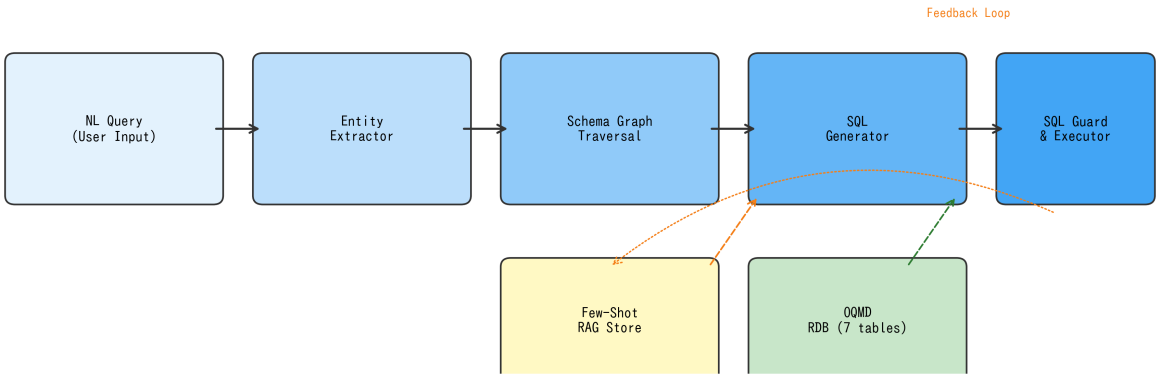

Figure 1: System architecture of the Schema-Graph-Assisted Text-to-SQL pipeline. Solid arrows indicate the primary data flow; dashed arrows indicate the Few-Shot RAG feedback loop (successful query pairs are stored and retrieved for future queries). The SQL Guard acts as a final safety layer before database execution.

2.1 Database Schema Design for Materials Data

2.1.1 Normalization of OQMD Data into a 7-Table Schema

OQMD's REST API returns flat JSON records (one entry = one JSON object), which we normalize into a 7-table relational schema (Fig. 2, Table 1). The design follows Boyce-Codd Normal Form (BCNF) and is motivated by the following materials-domain considerations:

Fig. 2: Entity-Relationship Diagram — Materials Database Schema (7 Tables)

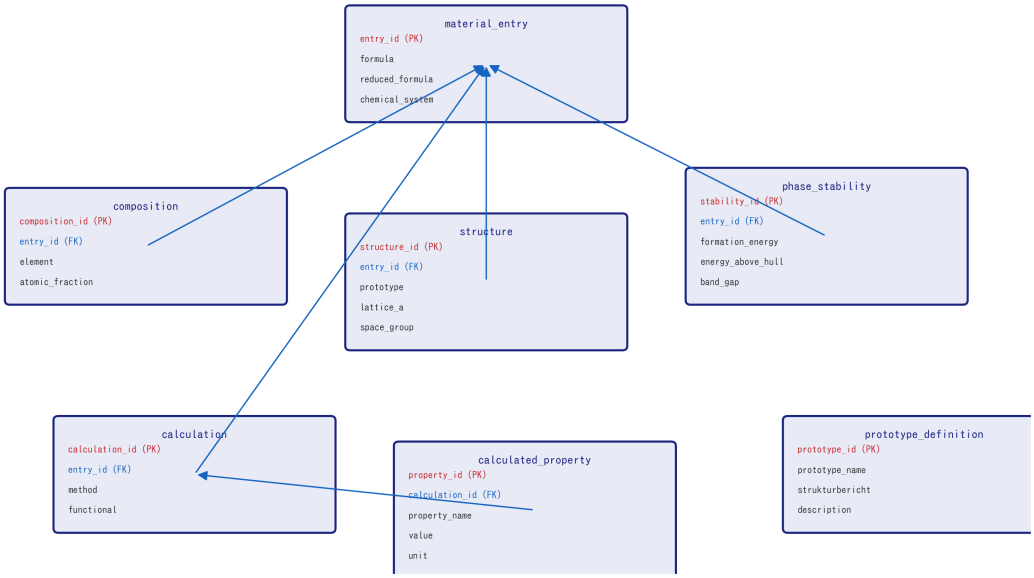

Figure 2: Entity-Relationship diagram of the 7-table materials database schema. All tables are connected to `material_entry` via foreign keys (blue lines). Red = primary key, Blue = foreign key.

Table 1: Database schema overview (7 tables).

Table	PK	FK	Key Columns
material_entry	entry_id	—	formula, reduced_formula, chemical_system
composition	composition_id	entry_id	element, atomic_fraction
structure	structure_id	entry_id	prototype, strukturbericht, lattice_a, space_group
phase_stability	stability_id	entry_id	formation_energy_per_atom, energy_above_hull, band_gap
calculation	calculation_id	entry_id	method, functional
calculated_property	property_id	calculation_id	property_name, value, unit
prototype_definition	prototype_id	—	prototype_name, strukturbericht, description

Composition table separation. A single compound may contain multiple elements (e.g., $Fe_3Al \rightarrow$ Fe and Al). By storing each element as a separate row in the `composition` table, we enable flexible element-based queries (AND, OR, NOT) without string parsing. This normalization, however, introduces a critical complexity: a query for "compounds containing both Ni and Al" cannot use a simple `WHERE element = 'Ni' AND element = 'Al'` (which would return zero rows, since no single row contains both elements simultaneously). Instead, `EXISTS` subqueries are required (see Section 2.3.4).

Structure/Phase stability/Calculation separation. Crystal structure information (prototype, lattice constant), thermodynamic stability (energy above hull), and calculation metadata (DFT functional) are stored in separate tables to allow independent updates and extensions.

Entity-Attribute-Value (EAV) for calculated properties. The `calculated_property` table uses an EAV pattern (`property_name`, `value`, `unit`) to accommodate arbitrary computed properties (bulk modulus, shear modulus, etc.) without schema modifications.

2.1.2 XML/YAML Representation for RAG Context Injection

The schema structure is serialized as YAML (`allowed_schema.yaml`), which serves dual purposes: (a) the SQL Guard's table/column whitelist (Section 2.6), and (b) RAG context for LLM prompts—the schema YAML is injected into the system message so the LLM understands available tables, columns, and their types. This approach is analogous to schema serialization techniques used in general T2SQL [Gao et al., 2023], but here specialized for materials databases with domain-specific column semantics (e.g., `energy_above_hull` represents E_{hull}).

2.2 Entity Extractor with Materials Terminology Dictionary

The entity extractor converts natural language queries into structured condition dictionaries using a YAML-based bilingual terminology dictionary (`material_terms.yaml`).

2.2.1 Dictionary Structure

The dictionary contains four categories:

1. **Prototype aliases**: Mapping between prototype names and their variants.
   Example: `B2` ↔ [`B2`, `CsCl`, `CsCl` 型, `B2` 型]
   `L12` ↔ [$\texttt{L1}_2$, `L12`, `AuCu3`, $\texttt{Cu}_3\texttt{Au}$ 型, γ']
2. **Element mappings**: Symbol ↔ Japanese/English name correspondence.
   Example: `Fe` ↔ [`Fe`, 鉄 `(tetsu)`, `iron`]
3. **Stability terms**: Natural language → numerical threshold.
   "stable" / "安定" → `energy_above_hull` ≤ 0.001 eV/atom
   "metastable" / "準安定" → `energy_above_hull` ≤ 0.05 eV/atom
4. **Property terms**: Mapping property descriptions to table.column references.
   "lattice constant" / "格子定数" → `structure.lattice_a`
   "formation energy" / "形成エネルギー" → `phase_stability.formation_energy_per_atom`

2.2.2 Unicode Normalization and Pattern Matching

Unicode subscript characters (☒☒☒☒☒☒☒☒☒☒) are normalized to ASCII digits before matching. Element symbol recognition uses word-boundary regular expressions to correctly distinguish standalone "Fe" from "FeCoNi" substrings.

2.2.3 Output Format

The extractor returns a structured dictionary:

Listing 1: Entity extraction example.

```
# Input: "Show stable B2 compounds containing Fe"
# Output:
{
  "prototype": "B2",
  "contains_elements": ["Fe"],
  "stability": "stable",
  "sort_by": None,
```

```
  "sort_order": None
}
```

2.3 Schema Graph Traversal Engine

This is the core algorithmic contribution of the proposed method. The engine represents the database schema as a graph and computes optimal JOIN paths automatically.

2.3.1 Graph Construction from PostgreSQL Metadata

Foreign key relationships are extracted dynamically from PostgreSQL's `information_schema`:

Listing 2: FK relationship extraction query.

```
SELECT tc.table_name AS source_table,
       kcu.column_name AS source_column,
       ccu.table_name AS target_table,
       ccu.column_name AS target_column
  FROM information_schema.table_constraints tc
  JOIN information_schema.key_column_usage kcu
    ON tc.constraint_name = kcu.constraint_name
  JOIN information_schema.constraint_column_usage ccu
    ON ccu.constraint_name = tc.constraint_name
 WHERE tc.constraint_type = 'FOREIGN KEY'
   AND tc.table_schema = 'public';
```

The result is stored as an undirected NetworkX [Hagberg et al., 2008] graph $G = (V, E)$ where $V = \{\text{table names}\}$ and $E = \{\text{FK relationships}\}$. Because FK metadata is read at runtime, the graph automatically adapts to schema changes without code modifications.

2.3.2 Minimal Covering JOIN Path: Steiner Tree Approximation

Given a set of *required tables* $R \subseteq V$ (determined by the entity extractor's output), we need the minimum subgraph of G that connects all tables in R. This is the Steiner tree problem in graphs [Hwang et al., 1992], which is NP-hard in general.

For our 7-node, 6-edge schema graph, we employ a greedy heuristic (Algorithm 1) that is exact for tree-structured graphs (which our FK graph is):

Complexity. For a graph with $|V| = n$ nodes and $|R| = k$ required tables, the algorithm performs $O(k^2)$ shortest-path computations, each $O(n + |E|)$ via BFS. For our schema ($n = 7$, $k \leq 5$), this completes in microseconds.

Correctness for tree-structured FK graphs. When G is a tree (as in our schema, where `material_entry` is the hub), the Steiner tree is unique and our greedy algorithm finds it exactly. For schemas with FK cycles (e.g., self-referencing tables), the approximation may include unnecessary edges, but this is acceptable for SQL generation (extra JOINs increase query cost but do not affect correctness if DISTINCT is applied).

2.3.3 JOIN Path → SQL FROM/JOIN Clause Generation

Each path in P is converted to SQL JOIN clauses. The FK metadata provides the join column names:

Listing 3: JOIN clause generation from path.

```
# Path: [material_entry, structure]
# FK: structure.entry_id -> material_entry.entry_id
```

Algorithm 1 Minimal covering JOIN subgraph (Steiner tree approximation).

Require: FK relationship graph $G = (V, E)$, required table set R
Ensure: List of JOIN paths P

$P \leftarrow \emptyset$, covered $\leftarrow \emptyset$
for all $(s, t) \in \binom{R}{2}$ **do**
  if $s \in$ covered $\wedge\ t \in$ covered **then**
    continue ▷ Both endpoints already connected
  end if
  $p \leftarrow \text{ShortestPath}(G, s, t)$ ▷ BFS on FK graph
  if $p \neq \emptyset$ **then**
    $P \leftarrow P \cup \{p\}$
    covered $\leftarrow$ covered $\cup$ nodes(p)
  end if
end for
for all $r \in R \setminus$ covered **do** ▷ Handle isolated required tables
  $c \leftarrow$ nearest node in covered
  $P \leftarrow P \cup \{\text{ShortestPath}(G, r, c)\}$
end for
return P

```
# Generated:
#   FROM material_entry m
#   JOIN structure s ON s.entry_id = m.entry_id
```

Table aliases are assigned automatically (first letter of table name, with conflict resolution for collisions).

2.3.4 Multi-Element AND Query: The EXISTS Subquery Pattern

A critical materials-specific challenge arises from the composition table normalization. Querying "compounds containing both Ni and Al" requires:

Listing 4: Multi-element AND query using EXISTS subqueries.

```
SELECT DISTINCT m.entry_id, m.formula
FROM material_entry m
WHERE EXISTS (
    SELECT 1 FROM composition
    WHERE entry_id = m.entry_id AND element = 'Ni')
  AND EXISTS (
    SELECT 1 FROM composition
    WHERE entry_id = m.entry_id AND element = 'Al')
LIMIT 100;
```

The Schema Graph engine detects when multiple elements are specified and automatically generates EXISTS subqueries instead of direct JOINs. Without this pattern (Naive approach), the query `WHERE c.element = 'Ni' AND c.element = 'Al'` returns zero rows because no single composition row can satisfy both conditions simultaneously. This is the **most critical failure mode** prevented by the Schema Graph method.

2.3.5 Comparison: Naive vs. Schema Graph (Worked Example)

Table 2 illustrates the difference for query "Show B2 compounds containing Fe":

Table 2: SQL generation comparison: Naive vs. Schema Graph for "B2 compounds containing Fe".

Aspect	Naive (Level 0)	Schema Graph (Level 1)
JOIN tables	All 6 tables always	Only composition + structure (2 tables)
JOIN count	5 JOINs	2 JOINs
DISTINCT	No → duplicate rows	Yes
LIMIT	No → unbounded	Yes (100)
Safety validation	No	Full (keyword + table whitelist)
Multi-element AND	Incorrect (0 rows)	Correct (EXISTS)

2.4 SQL Generation: Rule-based and LLM Modes

The system supports two SQL generation modes, both operating within the Schema Graph constraints:

2.4.1 Rule-based Mode (Level 1)

A deterministic template-based generator constructs SQL from the extracted conditions and Schema Graph JOIN paths. No external API is required; latency is <100 ms. Limitations: cannot generate numeric comparison WHERE clauses (e.g., `band_gap > 1.0`), cannot handle negation ("compounds *not* containing Fe"), and cannot understand terminology not in the dictionary.

2.4.2 LLM Mode with Few-Shot RAG (Level 2)

When the Rule-based mode's dictionary coverage is insufficient (e.g., unregistered elements, numeric conditions, complex phrasing), the system falls back to LLM-based generation. The LLM (GPT-5 in our experiments) receives a structured prompt containing:

1. **System message**: Task description + Schema YAML (table names, column names, types, FK relationships)
2. **Few-Shot examples**: Top-k similar NL–SQL pairs from the RAG store (Section 2.5)
3. **User message**: The natural language query
4. **Constraints**: "Generate only SELECT statements", "Include LIMIT 100", "Use only tables listed in the schema"

The generated SQL is then validated by the SQL Guard (Section 2.6) before execution.

2.5 Few-Shot RAG Store (SQL-as-Few-Shot-Examples)

2.5.1 Architecture

The Few-Shot store implements a RAG [Lewis et al., 2020] pattern specialized for T2SQL:

1. **Accumulation**: When a query successfully produces results, the (NL query, SQL, conditions, row count, source) tuple is appended to a JSON store.
2. **Retrieval**: For a new query, TF-IDF [Salton and Buckley, 1988] cosine similarity is computed against all stored NL queries, and the top-k (default $k = 3$) are returned.
3. **Injection**: Retrieved examples are formatted as "`Question: ... SQL: ...`" blocks and prepended to the LLM prompt.

2.5.2 TF-IDF Tokenization for Materials Queries

Standard NLP tokenizers are inadequate for materials queries because they split chemical formulas and element symbols incorrectly. Our tokenizer uses a combined regex pattern:

Listing 5: TF-IDF tokenizer for materials queries.

```
# Tokenization pattern:
# 1. Chemical symbols: [A-Z][a-z]? (Fe, Ni, Al, ...)
# 2. Japanese characters: [\u3040-\u9fff]+
# 3. English words: [a-z]+
# 4. Numbers: \d+
tokens = re.findall(
  r'[A-Z][a-z]?|[\u3040-\u9fff]+|[a-z]+|\d+',
  query.lower()
)
```

IDF is computed using $\text{IDF}(t) = \log(N/n_t)$ where N is the total number of stored examples and n_t is the number containing term t. Cosine similarity between query and stored example TF-IDF vectors determines ranking.

2.5.3 Seed Example Extraction from Research Papers

To bootstrap the Few-Shot store without manual curation, we implemented automatic seed example extraction from LaTeX manuscripts. The extractor scans for patterns such as:

- `B2[-s]type s*(formula)` → {prototype: B2, elements: extracted}
- `L1[$_{}2]+[-s]*(formula)` → {prototype: L12, elements: extracted}
- Context keywords within 100 characters: "lattice", "stable", "formation energy"

For each extracted compound mention, a canonical NL query and corresponding SQL are generated. In our experiments, 4 seed examples were automatically extracted from `hea_lattice_prediction.tex`, complementing 7 manually curated examples (11 total; Table 3).

Table 3: Few-Shot store composition (11 examples).

Source	Count	%	Example Query
Manual curation	7	64%	"Show B2 compounds containing Fe"
Paper extraction	4	36%	"Show properties of B2 FeAl"

2.6 SQL Guard: Multi-layer Safety Validation

All generated SQL—whether from Rule-based or LLM mode—passes through a 5-layer validation pipeline before database execution:

1. **Multiple statement detection**: Reject if semicolons appear within the SQL body (prevents piggyback attacks such as `SELECT ...; DROP TABLE ...`).
2. **Forbidden keyword detection**: Scan for INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE, GRANT, REVOKE, COPY using word-boundary regex matching.
3. **SELECT-only restriction**: Reject any statement not beginning with SELECT or WITH.
4. **Table whitelist verification**: Extract all table names referenced in FROM/JOIN clauses and verify each appears in `allowed_schema.yaml`. Queries referencing undeclared tables (e.g., `secret_passwords`) are rejected.
5. **Automatic LIMIT injection**: If no LIMIT clause is present, append `LIMIT 100` to prevent unbounded result sets from consuming excessive memory or computation.

Optionally, SQLGlot [Mao, 2023] performs AST-level syntax validation (detecting structurally malformed SQL that passes regex checks).

2.7 Recommended Operational Configuration: Cascading Mode

Based on the characteristics of each generation mode, we recommend a cascading strategy:

1. **Stage 1**: Attempt Rule-based generation (Level 1, <100 ms).
2. **Decision**: If extracted conditions are empty or dictionary coverage is below a threshold, escalate.
3. **Stage 2**: Fall back to LLM generation with Few-Shot RAG (Level 2, ∼7 s).
4. **Common**: SQL Guard validation is always applied regardless of generation mode.

This approach processes dictionary-covered queries (estimated 70–80% of typical materials queries) without external API dependency, reserving LLM calls for complex or novel queries.

3 Data Preparation

3.1 OQMD API Data Acquisition

Data were retrieved from the OQMD RESTful API (`https://oqmd.org/oqmdapi/formationenergy`) for two intermetallic prototype structures:

- **B2 (CsCl-type)**: Filter `prototype=CsCl`. 636 unique compositions after deduplication. Stable ($E_{\text{hull}} \leq 0.001$ eV/atom): 185. Metastable ($E_{\text{hull}} \leq 0.05$ eV/atom): 198.
- **$L1_2$ ($AuCu_3$-type)**: Filter `prototype=AuCu3`. 273 unique compositions. Stable: 88. Metastable: 138.

Total: 909 unique compounds. For each entry, six fields were retrieved: name (chemical formula), entry_id, prototype, delta_e (formation energy per atom), stability (energy above hull), and band_gap.

The API returns paginated JSON responses (`limit=200, offset=0,200,...`). Duplicate entries (same chemical formula, different entry_id from different DFT runs) were deduplicated by retaining the lowest-energy entry.

3.2 RDB Ingestion

Each OQMD entry is decomposed into the 7-table schema. For example, Fe_3Al ($L1_2$ prototype) generates: 1 row in `material_entry`, 2 rows in `composition` (Fe: 0.75, Al: 0.25), 1 row in `structure` (prototype=L12, lattice_a, space_group), 1 row in `phase_stability` (formation_energy, energy_above_hull, band_gap), and 1 row in `calculation` (method=GGA).

3.3 Schema Extension

The following columns were added to accommodate OQMD fields not in the original schema: `band_gap` (REAL) in `phase_stability` and `space_group` (TEXT) in `structure`. The materials terminology dictionary was updated with corresponding terms (band_gap, volume_per_atom, space_group).

4 Results

4.1 Test Case Design

39 test cases were designed across 6 categories (Table 4) to evaluate correctness, safety, and robustness.

4.2 Three-Level Comparison

Fig. 3 and Table 5 show the comparison across three system levels.

Table 4: Test case categories (39 total).

Category	IDs	n	Validation Focus
Normal	A01–A15	15	Element, prototype, stability, sort, compound conditions
No results	B01–B05	5	Rare/noble gas elements, non-existent combinations → expect 0 rows
Sloppy input	C01–C10	10	Empty input, irrelevant text, single-word, numeric conditions
Contradictory	D01–D02	2	"stable AND metastable", "Fe-free Fe compound"
SQL injection	E01–E05	5	DROP, DELETE, INSERT, piggyback attacks
Safety check	F01–F02	2	Direct SQL input (SELECT, UPDATE)

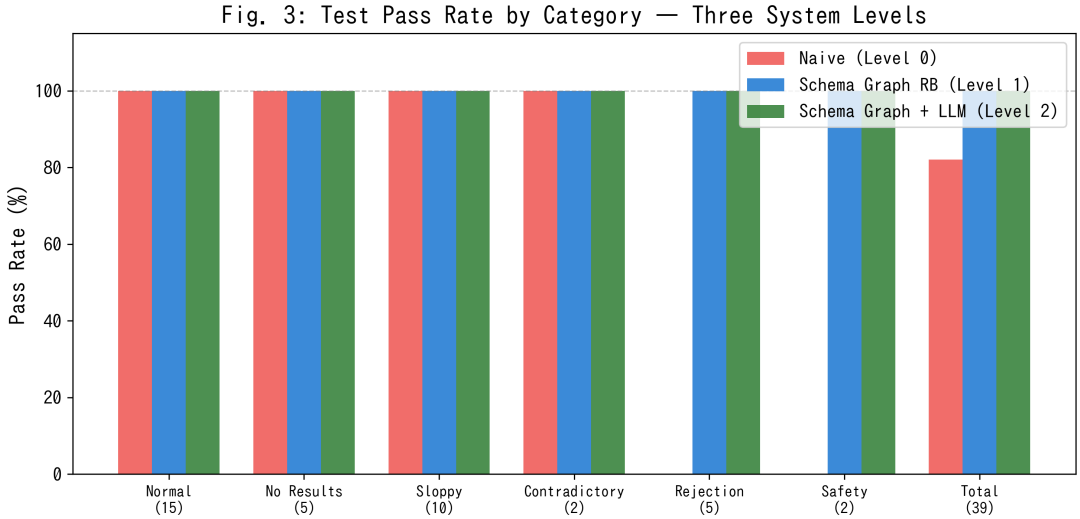

Figure 3: Test pass rate by category for three system levels. Naive (Level 0) fails on SQL injection and safety categories (no SQL Guard). Both Schema Graph levels (1 and 2) achieve 100% across all categories.

Table 5: Quantitative comparison of three system levels.

	Naive (L0)	SG + RB (L1)	SG + GPT-5 (L2)
Pass rate	32/39 (82%)	39/39 (100%)	39/39 (100%)
Mean latency	<100 ms	<100 ms	∼7,300 ms
Token consumption	0	0	49,103 (39 queries)
Unnecessary JOIN removal	No	Yes	Yes
Multi-element AND	Incorrect	Correct	Correct
SQL injection blocking	No	Yes	Yes
Unregistered term handling	Ignored	Ignored	Understood
Numeric WHERE generation	No	No	Yes
External API dependency	None	None	OpenAI

4.3 Rule-based vs. GPT-5 Detailed Comparison

4.3.1 Result Set Agreement (Jaccard Index)

For 32 comparable tests (excluding injection/safety categories), Jaccard similarity between Rule-based and GPT-5 result sets was computed (Fig. 4).

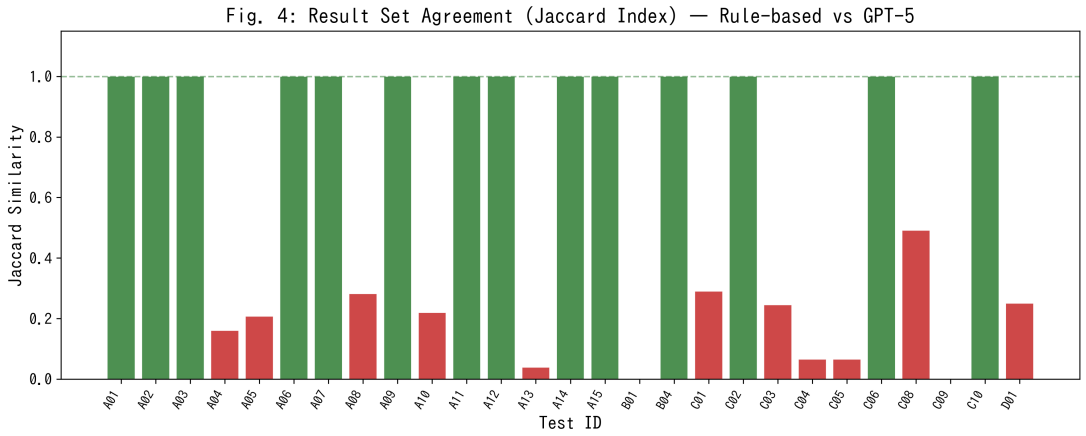

Figure 4: Jaccard index between Rule-based and GPT-5 result sets. Green: ≥ 0.8 (high agreement), Orange: 0.5–0.8 (moderate), Red: < 0.5 (low). Low-agreement cases primarily arise from GPT-5's superior semantic understanding of compound-specific queries (e.g., B01: Xe compounds, C09: NiAl specification).

4.3.2 Notable Divergences

Three cases illustrate where GPT-5 outperforms Rule-based:
B01: "Show B2 compounds containing Xe" Rule-based: Xe is not in the dictionary → condition ignored → returns all B2 compounds (100 rows, **false positives**). GPT-5: correctly generates `WHERE element = 'Xe'` → 2 rows (CsXe, MgXe).
C09: "NiAl L12" Rule-based: extracts Ni, Al, L12 independently → returns all $L1_2$ with Ni or Al (100 rows). GPT-5: understands "NiAl" as a specific compound → returns $AlNi_3$ only (1 row).
D02: "Fe-free Fe B2 compounds" Rule-based: cannot parse negation → returns Fe-containing B2 compounds. GPT-5: recognizes contradiction → generates appropriate query.

4.3.3 Latency and Cost

Fig. 6 shows per-query latency. Rule-based mode is consistently <100 ms. GPT-5 mode averages 7,305 ms (max 23,348 ms), a ∼70× increase. Total token consumption for 39 queries was 49,103 tokens (∼1,259 tokens/query).

4.4 OQMD API Ground Truth Validation (Integration Test)

For 8 test cases, we queried the OQMD REST API with equivalent filter conditions and compared the results against T2SQL output (Table 6, Fig. 7). Precision = |T2SQL ∩ OQMD| / |T2SQL|; Recall = |T2SQL ∩ OQMD| / |OQMD|.

Significance as an Integration Test. This comparison validates the **entire 5-stage data transformation pipeline**: (1) API JSON → CSV conversion, (2) CSV → 7-table normalization, (3) NL → SQL conversion (entity extraction + Schema Graph), (4) JOIN path correctness (FK graph traversal), and (5) SQL execution with DISTINCT/LIMIT. Precision = 1.0 confirms

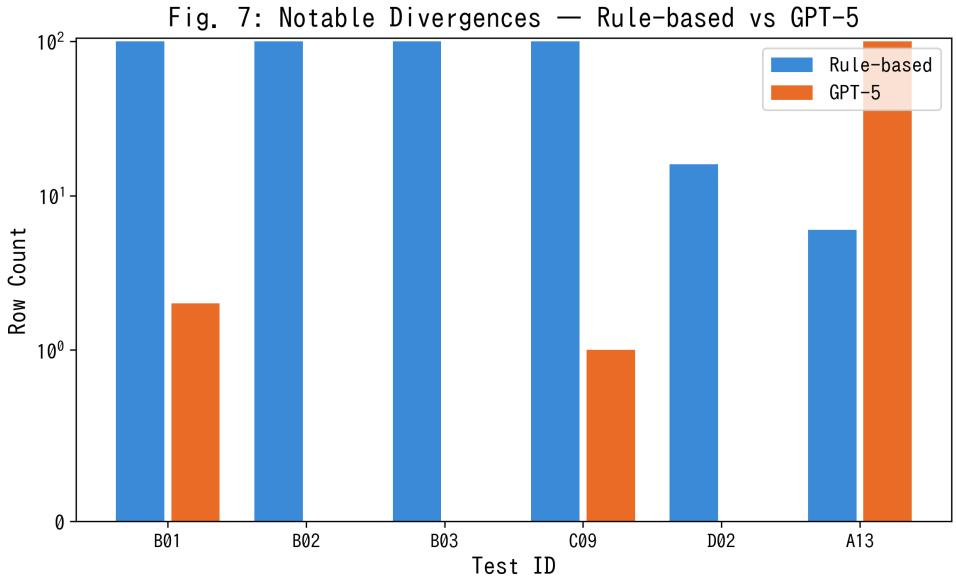

Figure 5: Row count comparison for notable divergence cases. B01–B03: Rule-based returns excessive results due to unregistered elements; GPT-5 returns correct (small) result sets.

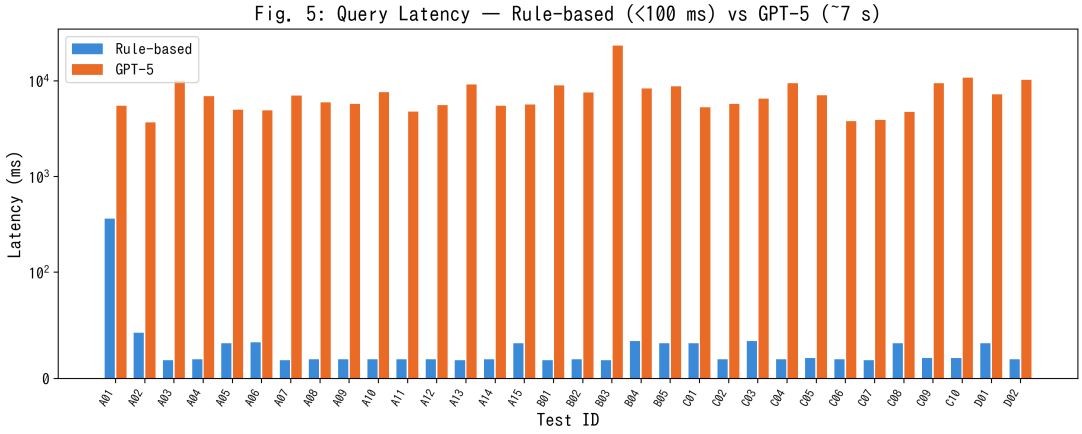

Figure 6: Per-query latency comparison (log scale). Rule-based: consistently <100 ms. GPT-5: ∼7.3 s average.

Table 6: OQMD API ground truth comparison (Precision / Recall).

ID	Query Condition	OQMD	T2SQL	Prec.	Rec.	Note
A01	Fe + B2	7	7	1.000	1.000	Exact match
A02	Stable $L1_2$ (sorted)	88	88	1.000	1.000	Exact match
A04	All B2	483	95	1.000	0.197	LIMIT 100
A05	Metastable B2	260	90	1.000	0.346	LIMIT 100
A06	Co + stable $L1_2$	1	1	1.000	1.000	Exact match
A10	All $L1_2$	273	100	1.000	0.366	LIMIT 100
A15	Ni + stable $L1_2$	6	6	1.000	1.000	Exact match
C08	Stable B2, band_gap > 0	0	78	0.000	—	API filter N/A

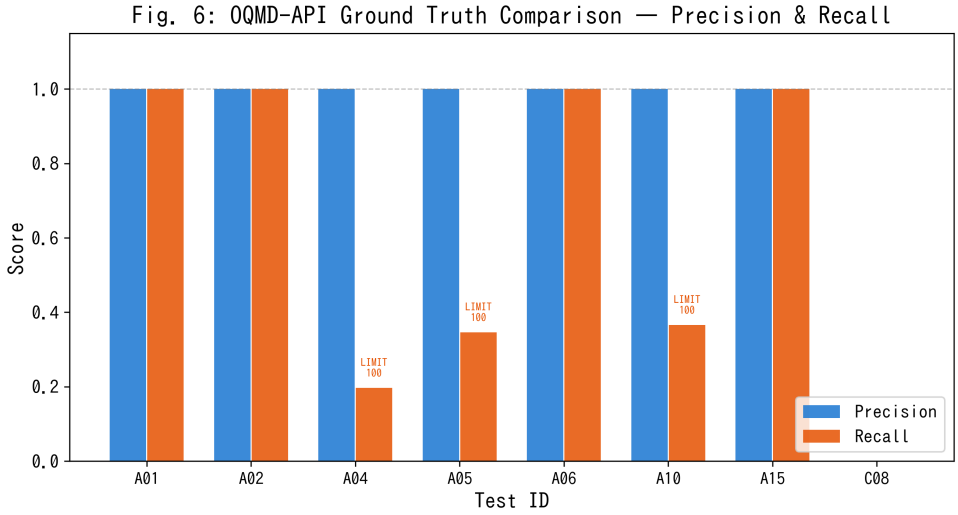

Figure 7: Precision (blue) and Recall (orange) against OQMD API ground truth. All tests except C08 achieve Precision = 1.0. Low Recall in A04/A05/A10 is due to LIMIT 100 constraint, not data inaccuracy.

that no data loss, corruption, or incorrect JOIN occurs across these five stages. If **any** stage contained a bug—e.g., incorrect element splitting in the composition table, FK integrity violations, or missing JOIN paths—Precision would fall below 1.0.

Note: because the same data source (OQMD) is used for both the database content and the ground truth, this validation does not constitute an independent evaluation of the system's generalization to unseen databases. Cross-database validation (e.g., against Materials Project) remains future work.

4.5 SQL Injection and Safety Results

All 7 adversarial inputs (E01–E05, F01–F02) were blocked by the SQL Guard (Table 7).

Table 7: SQL injection and safety test results (all blocked).

ID	Input	Blocking Reason
E01	`DROP TABLE material_entry;`	Forbidden keyword: DROP
E02	`SELECT *; DELETE FROM composition;`	Multiple statements + DELETE
E03	B2 compounds; `DROP TABLE structure;`	Multiple statements + DROP
E04	`SELECT * FROM secret_passwords`	Disallowed table
E05	`INSERT INTO material_entry ...`	Forbidden keyword: INSERT
F01	`SELECT entry_id FROM material_entry`	No LIMIT → auto-added
F02	`UPDATE material_entry SET ...`	Forbidden keyword: UPDATE

5 Discussion

5.1 Multi-dimensional Assessment

Fig. 8 presents a radar chart comparing the three system levels across six dimensions: accuracy, latency (inverse), cost (inverse), safety, vocabulary coverage, and numeric filter capability.

Fig. 8: Multi-dimensional Comparison of Three System Levels

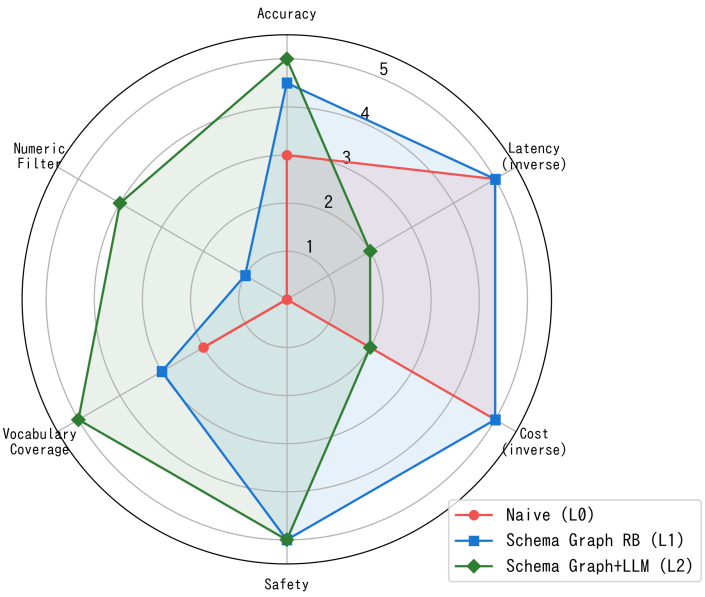

Figure 8: Multi-dimensional comparison of three system levels (5-point scale per axis). Schema Graph + LLM (Level 2) is most comprehensive but trades off latency and cost.

Table 8: Advantages and disadvantages of each system level.

Level	Advantages	Disadvantages
Naive (L0)	Minimal implementation (∼50 LOC). No dependencies. Fastest (<100 ms)	Unnecessary JOINs. Multi-element AND fails. No safety. No LIMIT/DISTINCT
SG + RB (L1)	No external API. <100 ms. Full safety. 100% deterministic. Reproducible	Silent failure on unregistered terms. No numeric WHERE. Manual vocabulary expansion
SG + LLM (L2)	Handles unregistered terms. Numeric/negation capable. Contextual understanding	External API dependency (cost, latency, privacy). Non-deterministic. Hallucination risk

5.2 Pros and Cons of Each Level

5.3 Limitations

5.3.1 Test Case Self-Referentiality (High Severity)

The 39 test cases were designed by the system developer who knows the extraction patterns. The 100% pass rate is therefore expected under controlled conditions. Blind testing with free-form queries from materials researchers is required for genuine accuracy assessment.

5.3.2 Few-Shot RAG Effect Unvalidated (High Severity)

The Few-Shot store's primary function—improving LLM SQL generation via similar example injection—was not isolated in an ablation study. Future work should compare LLM performance with and without Few-Shot examples on a held-out test set.

5.3.3 Silent Failure (Medium Severity)

When unregistered terms appear in Rule-based mode, conditions are silently dropped, producing overly broad results without user notification. Implementing a "condition coverage score" that warns users when their query terms were partially unrecognized would mitigate this risk.

5.3.4 Small-Scale Schema (Medium Severity)

Our 7-table, 909-record schema is far smaller than production databases (OQMD: >1M entries, potentially dozens of tables). Schema Graph traversal's Steiner tree approximation is exact for tree-structured graphs but may degrade for larger schemas with FK cycles. Scalability testing on 20+ table schemas with 100K+ records is needed.

5.3.5 Numeric Condition Limitation in Rule-based Mode (Medium Severity)

Rule-based mode cannot generate numeric comparison clauses (e.g., `band_gap > 1.0`), which are frequently needed in materials screening. A numeric condition parser (regex-based extraction of inequality operators and values) would address this without requiring LLM calls.

5.4 Comparison with Related Systems

5.5 Significance for AI for Science (AI4S)

The proposed method has direct implications for the AI4S paradigm in materials science. Table 10 maps each AI4S workflow stage to the role of the T2SQL method.

Critically, the Schema Graph architecture is **database-agnostic**: because FK relationships are introspected at runtime from PostgreSQL metadata (Section 2.3), the same engine can be deployed on any normalized materials database (Materials Project, AFLOW, NOMAD, or institutional databases) without code modifications—only the terminology dictionary needs domain adaptation. This portability is essential for AI4S, where heterogeneous data sources must be queried through a unified interface.

Furthermore, the cascading Rule-based → LLM architecture (Section 2.7) aligns with AI4S requirements for **reliability and cost efficiency**: routine queries are handled deterministically without external API calls, while complex queries leverage LLM capabilities. This hybrid approach ensures that AI4S pipelines can operate at scale without incurring prohibitive API costs or latency.

Table 9: Comparison with related materials NLP / T2SQL systems.

System	Schema-aware	Normalized RDB	Materials-specific	Few-Shot	Key Difference
Spider [Yu et al., 2018]	Yes	Yes	No	No	General-purpose benchmark
BIRD [Li et al., 2024]	Yes	Yes	No	No	Large-scale, real-world data
DAIL-SQL [Gao et al., 2023]	Yes	Yes	No	Yes	Skeleton-based selection
MKNA [Zhuang and Farimani, 2026]	No	No (API)	Yes	No	Agent-based, no SQL
OptiMat [Hu and Turlo, 2026]	No	No	Yes	No	On-demand DFT
MP MCP	Partial	No (API)	Yes	No	MCP protocol
This work	**Yes**	**Yes**	**Yes**	**Yes**	FK graph + materials dict

Table 10: Role of T2SQL in the AI4S materials discovery workflow.

AI4S Stage	Conventional Approach	With T2SQL Method
Hypothesis generation	Researcher formulates query manually	LLM agent generates NL query from hypothesis
Data retrieval	Write SQL or API calls manually	NL → SQL automatic conversion via Schema Graph
Screening / filtering	Iterative SQL refinement	Few-Shot RAG reuses successful query patterns
Result interpretation	Manual inspection of query results	Structured output with metadata (prototype, stability, properties)
Feedback / learning	Knowledge stays with individual researcher	Successful queries accumulate in Few-Shot store for reuse

5.6 Future Directions

1. **Multi-database extension**: Adapt Schema Graph to Materials Project, AFLOW schemas. Dynamic FK introspection already supports this architecturally.
2. **AI4S agent integration**: Embed the T2SQL method as a tool within autonomous materials discovery agents (e.g., via MCP protocol or function calling), enabling end-to-end hypothesis $\rightarrow$ data retrieval $\rightarrow$ analysis pipelines.
3. **Blind user study**: Collect free-form queries from $\geq$10 materials researchers.
4. **Few-Shot ablation study**: Quantify Few-Shot RAG impact on LLM SQL quality.
5. **Interactive query refinement**: Return clarification questions for ambiguous queries instead of silent fallback.
6. **Numeric condition parser**: Rule-based extraction of inequality conditions.
7. **Large-scale schema validation**: 20+ tables, 100K+ records.
8. **Cross-lingual embedding retrieval**: Replace TF-IDF with transformer embeddings for Few-Shot similarity, improving cross-lingual matching—a prerequisite for international AI4S collaboration where queries arrive in multiple languages.

6 Conclusion

We presented a Schema-Graph-Assisted Text-to-SQL method for materials databases, validated on 909 OQMD intermetallic compounds (B2 and $L1_2$ types). The method addresses a specific gap in materials informatics: **schema-aware SQL generation on normalized relational databases** with materials-domain vocabulary.

Key findings:

1. **Schema Graph traversal** eliminates unnecessary JOINs and correctly generates EXISTS subqueries for multi-element AND searches—the most critical failure mode in normalized composition tables.
2. **Rule-based mode** achieves 39/39 test pass rate at <100 ms latency with zero external API dependency, suitable for dictionary-covered queries.
3. **LLM mode (GPT-5)** extends coverage to unregistered terms, numeric conditions, and negation, at $\sim$70$\times$ latency cost.
4. **OQMD API ground truth comparison** confirms Precision = 1.0 for the full data pipeline (API $\rightarrow$ RDB $\rightarrow$ NL $\rightarrow$ SQL $\rightarrow$ results), validating integration correctness.
5. **SQL Guard** blocks all tested injection attacks and enforces result set bounds.

The cascading Rule-based $\rightarrow$ LLM architecture balances speed, cost, and coverage, enabling materials researchers to query normalized databases using natural language without SQL expertise.

In the broader context of **AI for Science (AI4S)**, this work addresses a critical infrastructure gap: the ability to reliably convert natural language intent into executable database queries is a prerequisite for autonomous AI-driven materials discovery workflows. As AI agents increasingly mediate between researchers and data, the Schema Graph T2SQL method provides a **portable, database-agnostic foundation** that can be deployed across heterogeneous materials databases (OQMD, Materials Project, AFLOW, institutional databases) via runtime FK introspection. The Few-Shot RAG feedback loop further enables **continuous improvement without retraining**, aligning with the self-improving nature of AI4S systems.

While the current validation is limited in scale and test case independence, the method establishes a reproducible framework for materials-database-specific T2SQL that can be extended to larger schemas, additional data sources, and integration into AI4S agent architectures.

Acknowledgements

DFT calculation data were obtained from the Open Quantum Materials Database (OQMD). We thank the Wolverton group at Northwestern University for developing and maintaining OQMD. This work was supported by NIMS.

References

Hanchen Wang, Tianfan Fu, Yuanqi Du, Wenhao Gao, Kexin Huang, Ziming Liu, Payal Chandak, Shengchao Liu, Peter Van Katwyk, Andreea Deac, Anima Anandkumar, Katja Bergen, Carla P. Gomes, Shirley Ho, Pushmeet Kohli, Joan Lasenby, Jure Leskovec, Tie-Yan Liu, Arjun Manber, Debapratim Misra, Eliot Moret, Joelle Pineau, Ben Poole, Marina Sacks, Saayan SenGupta, Jian Sun, Jie Tang, Adam Trischler, Max Welling, Yaochen Xie, and Marinka Zitnik. Scientific discovery in the age of artificial intelligence. *Nature*, 620:47–60, 2023. doi: 10.1038/s41586-023-06221-2.

James E. Saal, Scott Kirklin, Muratahan Aykol, Bryce Meredig, and Christopher Wolverton. Materials design and discovery with high-throughput density functional theory: The Open Quantum Materials Database (OQMD). *JOM*, 65:1501–1509, 2013. doi: 10.1007/s11837-013-0755-4.

Scott Kirklin, James E. Saal, Bryce Meredig, Alex Thompson, Jeff W. Doak, Muratahan Aykol, Stephan Rühl, and Christopher Wolverton. The Open Quantum Materials Database (OQMD): Assessing the accuracy of DFT formation energies. *npj Comput. Mater.*, 1:15010, 2015. doi: 10.1038/npjcompumats.2015.10.

Anubhav Jain, Shyue Ping Ong, Geoffroy Hautier, Wei Chen, William Davidson Richards, Stephen Dacek, Shreyas Cholia, Dan Gunter, David Skinner, Gerbrand Ceder, and Kristin A. Persson. Commentary: The Materials Project: A materials genome approach to accelerating materials innovation. *APL Mater.*, 1:011002, 2013. doi: 10.1063/1.4812323.

Stefano Curtarolo, Wahyu Setyawan, Gus L. W. Hart, Michal Jahnatek, Roman V. Chepulskii, Richard H. Taylor, Shidong Wang, Junkai Xue, Kesong Yang, Ohad Levy, Michael J. Mehl, Harold T. Stokes, Denis O. Demchenko, and Dane Morgan. AFLOW: An automatic framework for high-throughput materials discovery. *Comput. Mater. Sci.*, 58:218–226, 2012. doi: 10.1016/j.commatsci.2012.02.005.

Claudia Draxl and Matthias Scheffler. NOMAD: The FAIR concept for big data-driven materials science. *MRS Bull.*, 43:676–682, 2018. doi: 10.1557/mrs.2018.208.

Oleg N. Senkov, Daniel B. Miracle, Kevin J. Chaput, and Jean-Philippe Couzinie. Development and exploration of refractory high entropy alloys — A review. *J. Mater. Res.*, 33:3092–3128, 2018. doi: 10.1557/jmr.2018.153.

Easo P. George, Dierk Raabe, and Robert O. Ritchie. High-entropy alloys. *Nat. Rev. Mater.*, 4:515–534, 2019. doi: 10.1038/s41578-019-0121-4.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of EMNLP*, pages 3911–3921, 2018.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? A BIg Bench

for Large-Scale Database Grounded Text-to-SQL (BIRD). *Advances in Neural Information Processing Systems*, 36, 2024.

Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.

Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. Next-generation database interfaces: A survey of LLM-based text-to-SQL. *arXiv preprint arXiv:2406.08426*, 2024.

Karime Maamari, Fadhil Abubaker, Daniel Jaroslawicz, and Amine Mhedhbi. The death of schema linking? text-to-SQL in the age of well-reasoned language models. *arXiv preprint arXiv:2408.07702*, 2024.

Yu Song, Santiago Miret, and Bang Liu. MatSci-NLP: Evaluating scientific language models on materials science language tasks using text-to-schema modeling. In *Proceedings of the 61st Annual Meeting of the ACL*, pages 3621–3639, 2023.

Vaibhav Mishra, Somaditya Singh, Mohd Zaki, et al. LLAMAT: Large language models for materials science information extraction. *arXiv preprint arXiv:2411.00177*, 2024.

Genmao Zhuang and Amir Barati Farimani. From natural language to materials discovery: The Materials Knowledge Navigation Agent. *arXiv preprint arXiv:2602.11123*, 2026.

Yang Hu and Vladyslav Turlo. OptiMat Alloys: A FAIR end-to-end agent with living database for computational multi-principal alloy exploration. *arXiv preprint arXiv:2604.21850*, 2026.

Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX, 2008. Proceedings of the 7th Python in Science Conference (SciPy).

Frank K. Hwang, Dana S. Richards, and Pawel Winter. *The Steiner Tree Problem*. North-Holland, 1992.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Inf. Process. Manage.*, 24:513–523, 1988. doi: 10.1016/0306-4573(88)90021-0.

Toby Mao. SQLGlot: A python SQL parser, transpiler, optimizer, and engine. https://github.com/tobymao/sqlglot, 2023.

A Full Test Case List

ID	Category	Natural Language Query	Result
A01	normal	Show B2 compounds containing Fe	PASS
A02	normal	Show stable $L1_2$ compounds sorted by formation energy	PASS
A03	normal	Show compounds containing both Ni and Al	PASS
A04	normal	Show all B2 compounds	PASS
A05	normal	Show metastable B2 compounds	PASS
A06	normal	Show stable $L1_2$ compounds containing Co	PASS
A07	normal	Show B2 and $L1_2$ compounds containing Fe and Al	PASS
A08	normal	Show γ' compounds	PASS
A09	normal	ニッケルを含む $L1_2$ 型化合物を出して	PASS
A10	normal	Show all $L1_2$ compound data	PASS
A11	normal	Show B2 compounds containing Ti sorted by lattice constant (desc)	PASS
A12	normal	Show CsCl-type compounds	PASS
A13	normal	Show Cu_3Au-type compounds	PASS
A14	normal	鉄を含む B2 化合物を出して	PASS
A15	normal	Show stable $L1_2$ compounds containing Ni	PASS
B01	no_results	Show B2 compounds containing Xe	PASS
B02	no_results	Show $L1_2$ compounds containing U and Pu	PASS
B03	no_results	Show B2 compounds containing Rn and Og	PASS
B04	no_results	Show stable B2 compounds containing Sc and Ir	PASS
B05	no_results	Show stable B2 compounds containing Pt and Ga	PASS
C01	sloppy	Show stable compounds	PASS
C02	sloppy	B2	PASS
C03	sloppy	Show something stable	PASS
C04	sloppy	(empty input)	PASS
C05	sloppy	What's the weather today?	PASS
C06	sloppy	Fe	PASS
C07	sloppy	lattice constant	PASS
C08	sloppy	Stable B2 compounds with large band gap	PASS
C09	sloppy	NiAl L12	PASS
C10	sloppy	B2 or L12 containing Fe	PASS
D01	contradictory	Stable AND metastable B2 compounds	PASS
D02	contradictory	Fe-free Fe B2 compounds	PASS
E01	rejection	DROP TABLE material_entry;	PASS

ID	Category	Natural Language Query	Result
E02	rejection	SELECT *; DELETE FROM composition;	PASS
E03	rejection	B2 compounds; DROP TABLE structure;	PASS
E04	rejection	SELECT * FROM secret_passwords	PASS
E05	rejection	INSERT INTO material_entry VALUES (...)	PASS
F01	safety	SELECT entry_id FROM material_entry	PASS
F02	safety	UPDATE material_entry SET formula='X'	PASS

B Generated SQL Examples

B.1 A01: "Show B2 compounds containing Fe"

Naive (Level 0):

```sql
SELECT m.entry_id, m.formula, s.prototype
FROM material_entry m
    JOIN composition c ON c.entry_id = m.entry_id
    JOIN structure s ON s.entry_id = m.entry_id
    JOIN phase_stability ps ON ps.entry_id = m.entry_id
    JOIN calculation calc ON calc.entry_id = m.entry_id
    JOIN calculated_property cp
      ON cp.calculation_id = calc.calculation_id
WHERE s.prototype = 'B2' AND c.element = 'Fe';
-- Issues: 3 unnecessary JOINs, no LIMIT, no DISTINCT
```

Schema Graph + Rule-based (Level 1):

```sql
SELECT DISTINCT m.entry_id, m.formula,
    s.prototype, s.lattice_a, s.space_group
FROM material_entry m
    JOIN composition c ON c.entry_id = m.entry_id
    JOIN structure s ON s.entry_id = m.entry_id
WHERE (s.prototype = 'B2' OR s.strukturbericht = 'B2')
    AND c.element = 'Fe'
LIMIT 100;
-- Optimal: 2 JOINs only, DISTINCT, LIMIT
```

Schema Graph + GPT-5 (Level 2):

```sql
SELECT DISTINCT m.entry_id, m.formula,
    s.prototype, s.lattice_a, s.space_group
FROM material_entry m
JOIN structure s ON s.entry_id = m.entry_id
JOIN composition c ON c.entry_id = m.entry_id
WHERE (s.prototype = 'B2' OR s.strukturbericht = 'B2')
  AND c.element = 'Fe'
LIMIT 100;
-- LLM generates equivalent SQL within Schema Graph constraints
```

B.2 A03: "Show compounds containing both Ni and Al"

Naive (Level 0):

```sql
SELECT m.entry_id, m.formula
FROM material_entry m
    JOIN composition c ON c.entry_id = m.entry_id
    ...(all table JOINs)...
WHERE c.element = 'Ni' AND c.element = 'Al';
-- FATAL: Single row cannot satisfy both conditions
-- -> Always returns 0 rows
```

Schema Graph + Rule-based (Level 1):

```sql
SELECT DISTINCT m.entry_id, m.formula
FROM material_entry m
WHERE EXISTS (
    SELECT 1 FROM composition
    WHERE entry_id = m.entry_id AND element = 'Ni')
  AND EXISTS (
    SELECT 1 FROM composition
    WHERE entry_id = m.entry_id AND element = 'Al')
LIMIT 100;
-- Correct multi-element AND via EXISTS subqueries
```